Appendix: Memory Type-Aware Oblivious Access Control Mechanism for LLM Multi-Agent Shared Memory

Zhen Hong[1]

Hainan University, Hainan, China
202330014500hainanu.edu.cn

1 Related Work and Comparative Analysis

Existing work can be examined from three dimensions: memory type awareness, privacy protection, and cryptographic enforcement. The core observation is: existing schemes satisfy at most one of these dimensions, while all three must be simultaneously satisfied to solve the security problem of cross-department shared memory—type awareness ensures the expressiveness of differentiated protection policies, privacy protection prevents the verification process itself from becoming an information leakage channel, and cryptographic enforcement guarantees that policies cannot be bypassed by prompt injection or other software-layer attacks.

Table 1. Technical feature comparison of schemes.

Scheme	Privacy	Type	Anti-Inj.	Delegation	Write
Ours	ZKP	3-layer	Strong (100%)	R1–R4 chain	Provenance
AIP [12]	Path exposed	None (flat)	Medium (sig.)	IBCT chain	None
A-MemGuard [11]	Plaintext	None	Weak (content)	None	Post-hoc
Collab. Mem. [3]	Partition	None	Weak	None	None
MemGPT	Plaintext	2-layer	Weak	None	None
LangChain	Plaintext	None	Weak	None	None
ABAC [7]	Plaintext attr.	1-dim label	Weak	Policy deleg.	None
RBAC [6]	Plaintext role	None	Weak	Role inherit.	None
Anon. Cred. [15,16]	Selective disc.	None	N/A	None	None

The three dimensions are indispensable: without type awareness, one cannot express "episodic memory is prohibited from cross-department propagation but procedural memory is allowed"; without privacy protection, the verification process itself leaks access patterns and department affiliation; without cryptographic enforcement, policies can be bypassed by prompt injection (our experiments prove 55% of attacks can bypass LLM self-constraints). Our scheme is the first to simultaneously satisfy all three dimensions in a unified framework, at the cost of 2.9-second proof generation latency for first access.

1.1 Multi-Agent Memory Security

Memory poisoning has been proven to be a serious threat to multi-agent systems. AgentPoison [8] achieves over 80% attack success rate by injecting adversarial text into RAG knowledge bases; BadAgent [9] demonstrates that backdoor agents can execute malicious writes under trigger conditions; Wang et al. [14] systematically reveal privacy leakage risks in LLM Agent memory.

A-MemGuard [11] is the first memory poisoning defense framework, detecting malicious content after writing through LLM semantic judgment. Its defense has

two structural limitations: adversarial text can bypass LLM detection (inherent weakness of semantic judgment), and detection occurs after writing (malicious content has already entered memory). These two limitations are not engineering defects but inherent boundaries of content-layer defense—it answers "is the written content malicious" rather than "who has the right to write." Our paper provides complementary defense from the permission control level, verifying the writer's authorization status through cryptographic means before writing.

1.2 Multi-Agent Shared Memory and Collaboration Frameworks

Collaborative Memory [3] is the first to study dynamic access control for multi-user LLM Agent memory sharing, but its scheme is implemented in plaintext, does not distinguish memory types, and does not support cross-department delegation. MemoryScope [10] proposes a single-agent three-layer memory classification architecture, providing a foundation for cognitive semantic modeling of memory, but does not involve security mechanism design.

The fundamental limitation of existing work is: Collaborative Memory focuses on "who can access which partition" but not "how different types of memory within that partition should receive different protection." This type-unaware design is particularly dangerous in cross-department scenarios—the same partition may simultaneously contain shareable semantic knowledge and non-disclosable episodic records.

1.3 Agent Delegation and Identity Protocols

AIP [12] proposes Invocation-Bound Capability Tokens (IBCTs), achieving efficient delegation chain verification (latency only 0.049ms). However, AIP's scope is designed as a flat tool list—it can express "which APIs this agent can invoke" but cannot express "which type of memory this agent can access, under what conditions, with what sanitization." More critically, AIP's delegation chain verification is executed in plaintext, allowing the verifier to completely track the delegation path and agent identity. IETF AIMS [4] and AAP [5] are standardizing cross-organization authentication and authorization frameworks but have not yet addressed memory type semantics and zero-knowledge verification.

The two are not substitutes but complements on the security requirements spectrum: AIP is suitable for high-frequency low-sensitivity tool invocation scenarios within organizations, while our scheme is suitable for cross-department high-sensitivity memory access scenarios.

1.4 Zero-Knowledge Proofs in Identity and Access Control

Groth16 [13] is currently the most widely deployed zkSNARK scheme, known for its 192-byte constant proof size and millisecond-level verification time. zkLogin [17] applies Groth16 to blockchain identity authentication with approximately 65,000 R1CS constraints and proof generation of approximately 3–4 seconds. Anonymous credential systems [15,16] achieve selective attribute disclosure but do not address memory type semantics.

Our choice of Groth16 is based on scenario characteristics: enterprise internal deployment makes trusted setup controllable (consortium-style MPC Setup),

minimal proof size is suitable for network transmission, and fastest verification time meets online requirements. Our circuit scale (40,043 constraints) is approximately 62% of zkLogin, well within Groth16's efficient usable range.

1.5 Comparative Summary

Our scheme achieves qualitative breakthroughs across three core dimensions: zero-knowledge privacy protection, cognitive semantics-driven type awareness, and cryptographic-level enforcement. Traditional schemes (RBAC, ABAC) and existing multi-agent memory systems all use plaintext verification; AIP uses HMAC signatures but the verifier can see the complete delegation path; our scheme encodes R1–R4 rules as circuit constraints, with policy execution guaranteed by mathematical laws. Additionally, our scheme uses Nullifiers for anti-replay, threshold Escrow for traceability, and Pedersen commitments for write provenance proof, forming a complete security guarantee chain. Emerging frameworks (MemGPT, LangChain) are designed for single-agent context management rather than security control; A-MemGuard focuses on content-layer defense, orthogonally complementary to our permission-layer defense.

2 Security Proof Supplements

2.1 Complete Proof of Theorem 1

Theorem 1 (Obliviousness). Under TA-2 assumptions, Π_{delegate} and Π_{write} satisfy obliviousness.

Proof. For Π_{delegate}, we construct a simulator $\mathsf{Sim}_{\text{del}}$ that, given only public inputs $(h_c, \mathsf{pk}_{\mathcal{I}}, \mathsf{nf})$, produces proofs computationally indistinguishable from real proofs. By the zero-knowledge property of Groth16 under the Generic Group Model on BN254, no PPT distinguisher can differentiate simulated from real proofs with non-negligible advantage. For Π_{write}, an analogous simulator Sim_{wr} operates on public inputs $(C_w, h_w, \mathsf{nf}_{\text{write}})$, and the Pedersen commitment C_w additionally provides information-theoretic hiding: for any id_w, the distribution of C_w is uniform over the group, independent of the committed identity. Thus, even an unbounded verifier gains no information about the writer's identity.

2.2 Complete Proof of Theorem 2

Theorem 2 (Unforgeability). Under TA-2 assumptions, protocol Π_{delegate} satisfies unforgeability.

Proof. Through reduction to EdDSA's EUF-CMA security. Assume there exists a PPT attacker $\mathcal{A}$ that can forge proofs with non-negligible probability ϵ; construct PPT reduction algorithm $\mathcal{B}$ as follows:

$\mathcal{B}$ receives EdDSA challenge public key pk^*, runs Groth16.Setup to generate proof system parameters $(\mathsf{pk}_{\text{Groth}}, \mathsf{vk}_{\text{Groth}})$, and publishes pk^* as the issuer public key.

Simulating signing oracle: When $\mathcal{A}$ queries the signature of token cap_i, $\mathcal{B}$ forwards cap_i to the EdDSA signing oracle, obtains signature σ_i, and returns it to $\mathcal{A}$.

Attacker output: $\mathcal{A}$ outputs forged proof (x^*, π^*), where $x^* = (h_c^*, \mathsf{pk}^*, \mathsf{nf}^*)$.

Knowledge extraction: $\mathcal{B}$ uses Groth16 knowledge extractor $\mathcal{E}$ (exists under GGM) to extract witness $w^* = (\mathsf{cap}_p^*, \mathsf{cap}_c^*, \sigma_p^*, \ldots)$ from π^*.

Forged signature: Since $\mathsf{scope}(x^*) \not\subseteq \mathsf{scope}(\mathsf{cap}_j)$ for all queried tokens cap_j, cap_p^* must be an unqueried token. $\mathcal{B}$ outputs $(\mathsf{cap}_p^*, \sigma_p^*)$ as an EdDSA forgery.

Success probability:

$$\mathsf{Adv}_{\mathcal{B}}^{\text{EUF-CMA}} \geq \mathsf{Adv}_{\mathcal{A}}^{\text{forge}} \cdot \Pr[\text{extraction succeeds}] - \mathsf{negl}(\lambda) \tag{1}$$

where $\Pr[\text{extraction succeeds}] \geq 1 - \mathsf{negl}(\lambda)$ (guaranteed by Groth16 knowledge soundness). Therefore, if $\mathcal{A}$ can forge proofs with non-negligible probability, then $\mathcal{B}$ can break EdDSA with non-negligible probability, contradiction.

2.3 Complete Proof of Theorem 3

Theorem 3 (Memory Type Safety). Under TA-2 assumptions, Π_{delegate} satisfies memory type safety: cross-department delegated episodic memory must undergo sanitization.

Proof. Assume for contradiction that a valid proof π^* exists where the witness contains $\mathsf{cross\_org} = 1$, $\mathsf{is\_ep}_i = 1$, and $\mathsf{sanitized}_i = 0$ for some partition i. Then the G4.2 constraint evaluates to $1 \times 1 \times (1 - 0) = 1 \neq 0$, which is unsatisfiable in the R1CS system. By the soundness of Groth16 on BN254 under the Generic Group Model, no PPT prover can produce an accepting proof for an unsatisfiable constraint system with non-negligible probability.

2.4 Complete Proof of Theorem 4

Theorem 4 (Collusion Resistance). Under TA-2 assumptions, Π_{delegate} is secure under $\mathsf{Game}_{\text{collusion}}(\lambda, t)$.

Proof. Define the security game:

- *Setup:* Challenger generates issuer key pair $(\mathsf{sk}_{\mathcal{I}}, \mathsf{pk}_{\mathcal{I}})$.
- *Token queries:* Attacker adaptively queries t tokens $\{\mathsf{cap}_1, \ldots, \mathsf{cap}_t\}$.
- *Challenge:* Attacker outputs (x^*, π^*).
- *Win condition:* $\mathsf{Verify}(\mathsf{vk}, x^*, \pi^*) = 1$ and $\mathsf{scope}(x^*) \not\subseteq \mathsf{scope}(\mathsf{cap}_i)$ for all $i \in [t]$.

By G6 holder binding, the witness must include sk^* such that $\mathsf{pk}^* = \mathsf{sk}^* \cdot G$, where pk^* is the holder field of cap_p^*, preventing reuse of another colluder's token without their private key. Using the knowledge extractor to extract witness $w^* = (\mathsf{cap}_p^*, \mathsf{cap}_c^*, \sigma_p^*, \ldots)$ from π^*:

Case 1: $\mathsf{cap}_p^* \in \{\mathsf{cap}_1, \ldots, \mathsf{cap}_t\}$. By G3 constraint $\mathsf{scope}_c^* \subseteq \mathsf{scope}_p^*$, therefore $\mathsf{scope}(x^*) \subseteq \mathsf{scope}(\mathsf{cap}_i)$ for some i, contradicting the win condition.

Case 2: $\mathsf{cap}_p^* \notin \{\mathsf{cap}_1, \ldots, \mathsf{cap}_t\}$. Then $(\mathsf{cap}_p^*, \sigma_p^*)$ is an EdDSA forgery, reducing to Theorem 2.

Therefore:

$$\Pr[\mathcal{A} \text{ wins}] \leq \mathsf{Adv}_{\mathcal{B}}^{\text{EUF-CMA}} + \mathsf{negl}(\lambda) \leq \mathsf{negl}(\lambda) \tag{2}$$

2.5 Complete Proof of Theorem 5

Theorem 5 (Write Provenance and Traceability). Under TA-2 assumptions, Π_{write} satisfies SG-5 (provenance trustworthiness) and SG-6 (traceability).

Proof. (a) Provenance trustworthiness (SG-5). We show that any valid proof π_{write} implies the writer holds a token authorizing the write. Suppose adversary $\mathcal{A}$ outputs an accepting proof π^* with public input $(C_w^*, h_w^*, \mathsf{nf}_{\text{write}}^*)$ without holding a write-permitted token. By Groth16 knowledge soundness (under GGM on BN254), there exists an extractor $\mathcal{E}$ that recovers a witness $w^* = (\mathsf{cap}^*, \sigma^*, \mathsf{id}_w^*, r_{\text{ped}}^*, \ldots)$ such that:

- W1 holds: σ^* is a valid EdDSA signature on cap^* under $\mathsf{pk}_{\mathcal{I}}$;
- W2 holds: the write permission bit in cap^* is set;
- W5 holds: $C_w^* = g^{\mathsf{id}_w^*} \cdot h^{r_{\text{ped}}^*}$.

If cap^* was never issued, then $(\mathsf{cap}^*, \sigma^*)$ constitutes an EdDSA forgery, contradicting EUF-CMA security of EdDSA on BabyJubJub. Therefore, cap^* must be a legitimately issued token whose holder has write authorization.

(b) Traceability (SG-6). Suppose, after the threshold Escrow recovers $(\mathsf{id}_w, r_{\text{ped}})$, the writer claims a different identity $\mathsf{id}'_w \neq \mathsf{id}_w$ with corresponding randomness r'_{ped} such that $C_w = g^{\mathsf{id}'_w} \cdot h^{r'_{\text{ped}}}$. Combining with $C_w = g^{\mathsf{id}_w} \cdot h^{r_{\text{ped}}}$:

$$g^{\mathsf{id}_w - \mathsf{id}'_w} = h^{r'_{\text{ped}} - r_{\text{ped}}}. \tag{3}$$

Since $\mathsf{id}_w \neq \mathsf{id}'_w$, we obtain

$$\log_g h = (\mathsf{id}_w - \mathsf{id}'_w) \cdot (r'_{\text{ped}} - r_{\text{ped}})^{-1} \bmod q, \tag{4}$$

which solves the discrete logarithm problem and contradicts TA-2. Hence the recovered identity is unique, establishing traceability.

2.6 Complete Proof of Theorem 6

Theorem 6 (Composition Security). The sequential composition of Π_{delegate} and Π_{write} preserves all security objectives SG-1 through SG-6.

Proof. We use a hybrid argument over three games:

- H_0: the real game where both protocols execute with real witnesses.
- H_1: identical to H_0 except that all transcripts of Π_{delegate} are produced by the Groth16 simulator $\mathsf{Sim}_{\text{del}}$ on public inputs only.
- H_2: identical to H_1 except that (i) all transcripts of Π_{write} are produced by the Groth16 simulator Sim_{wr}, and (ii) the Pedersen commitment C_w is replaced by a uniformly random group element.

By Groth16 zero-knowledge under the Generic Group Model on BN254,

$$|\Pr[\mathcal{D}(H_0) = 1] - \Pr[\mathcal{D}(H_1) = 1]| \leq \mathsf{Adv}^{\mathsf{zk}}_{\text{Groth16}}(\lambda) \leq \mathsf{negl}(\lambda). \tag{5}$$

Similarly, by Groth16 zero-knowledge plus the information-theoretic hiding of Pedersen commitments,

$$|\Pr[\mathcal{D}(H_1)=1]-\Pr[\mathcal{D}(H_2)=1]| \le \mathsf{Adv}^{\mathrm{zk}}_{\mathrm{Groth16}}(\lambda)+0 \le \mathsf{negl}(\lambda), \tag{6}$$

where the hiding term is exactly zero (information-theoretic). By transitivity, $|\Pr[\mathcal{D}(H_0)=1]-\Pr[\mathcal{D}(H_2)=1]| \le \mathsf{negl}(\lambda)$, so SG-1 (obliviousness) is preserved under composition.

Cross-protocol replay resistance. Suppose an adversary submits a proof π generated for Π_{delegate} as if it were a Π_{write} proof (or vice versa). The two protocols use disjoint nullifier spaces:

$$\mathsf{nf}_{\mathrm{delegate}} = \mathsf{Poseidon}(\text{“delegate”}\|\cdot), \quad \mathsf{nf}_{\mathrm{write}} = \mathsf{Poseidon}(\text{“write”}\|\cdot). \tag{7}$$

A collision $\mathsf{nf}_{\mathrm{delegate}} = \mathsf{nf}_{\mathrm{write}}$ would constitute a Poseidon collision on distinct prefixes, contradicting Poseidon collision resistance under TA-2. Furthermore, the public input structures and verification keys $(\mathsf{vk}_{\mathrm{del}}, \mathsf{vk}_{\mathrm{wr}})$ differ, so a proof accepted by $\mathsf{Verify}(\mathsf{vk}_{\mathrm{del}}, \cdot)$ cannot be accepted by $\mathsf{Verify}(\mathsf{vk}_{\mathrm{wr}}, \cdot)$ except with probability $\mathsf{Adv}^{\mathrm{sound}}_{\mathrm{Groth16}}(\lambda) \le \mathsf{negl}(\lambda)$.

The remaining objectives SG-2 through SG-6 are preserved because each is established within a single protocol and the composition does not introduce new oracle queries beyond those already accounted for in Theorems 2–5.

3 Circuit Implementation Parameters

3.1 Cryptographic Primitive Parameters

Poseidon Hash: BN254 scalar field $\mathbb{F}_p$, state width $t = 6$, full rounds $R_F = 8$, partial rounds $R_P = 57$, S-box $x \mapsto x^5$.

EdDSA Signature: BabyJubJub curve, $ax^2 + y^2 = 1 + dx^2y^2$ ($a = 168700$, $d = 168696$), signature algorithm EdDSA-BabyJubJub.

Groth16: BN254 pairing curve, $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$, proof size 192 bytes ($\pi = (A \in \mathbb{G}_1, B \in \mathbb{G}_2, C \in \mathbb{G}_1)$), verification key approximately 2KB.

3.2 G4 Constraint Detailed Breakdown

Table 2. G4 sub-constraint breakdown.

Sub-constraint	Function	Count	Implementation
G4.1.1	Policy encoding	64	Bit vector to integer
G4.1.2	Range proof	128	Comparison circuit
G4.2.1	Department determination	200	Non-zero check
G4.2.2	Type determination	116	Bit vector matching
G4.2.3	Label check	200	Ternary multiplication
G4.3.1	Sensitivity quantization	800	8-bit fixed point
G4.3.2	Integer division	1,200	Long division circuit
G4.3.3	Ceiling	320	Conditional addition
G4.3.4	Range proof	400	Comparison circuit

3.3 Default Protocol Parameters

Table 3. Default protocol parameters.

Parameter	Default	Description
Max delegation depth	6	Limits delegation chain length
Session token validity	5 min	Balances security and usability
Sensitivity amplification k	2	R3 rule parameter
Nullifier set capacity	2^{20}	Approx. 1 million entries
Revocation list capacity	2^{16}	Approx. 65,000 entries
Recommended $\hat{s}_i$ range	$[25, 243]$	Avoids boundary precision issues
Recommended depth limit	≤ 16	Avoids integer overflow

4 Experimental Supplementary Data

4.1 Experimental Setup

Hardware Platform: Intel Xeon Platinum 8369B @ 2.70GHz (8 cores), 14.7GB RAM, Ubuntu 20.04.

Software Stack: Circom 2.1.6, SnarkJS 0.7.x, BN254 curve, Node.js 20.x + TypeScript, LangGraph framework, GPT-4o-mini.

Code Availability: Circom circuit implementation, experimental datasets, and reproduction scripts are open-sourced at: https://github.com/[anonymous]/maas-ppac.

4.2 Prompt Injection Attack Detailed Results

Table 4. Detailed prompt injection attack results.

ID	Attack Type	LLM	ZKP	Result
1	Role-playing	✗	✓	Blocked (episodic)
2	Emergency pressure	✗	✓	Blocked (episodic)
3	Code block hiding	✓	—	LLM rejected
4	Multi-turn induction	✓	—	LLM rejected
5	Privilege escalation	✗	✓	Blocked (episodic)
6	System cmd disguise	✓	—	LLM rejected
7	Debug mode claim	✗	✓	Blocked (episodic)
8	Admin impersonation	✗	✓	Blocked (episodic)
9	Security check bypass	✓	—	LLM rejected
10	Context injection	✗	✓	Blocked (episodic)
11	Instruction override	✓	—	LLM rejected
12	Exception exploit	✓	—	LLM rejected
13	Log access request	✗	✗	Allowed (procedural)
14	Metadata query	✗	✗	Allowed (metadata)
15	Cross-dept access	✗	✓	Blocked (episodic)
16	History record query	✗	✓	Blocked (episodic)
17	Config file read	✓	—	LLM rejected
18	Sensitive data export	✗	✓	Blocked (episodic)
19	Batch query	✓	—	LLM rejected
20	Time window exploit	✓	—	LLM rejected

Key Observation: All 9 attacks attempting to access episodic memory (ID 1, 2, 5, 7, 8, 10, 15, 16, 18) were intercepted by ZKP. The 2 "leaks" (ID 13, 14) accessed

low-sensitivity procedural memory and metadata, which are policy-permitted for cross-department access and do not constitute security incidents.

4.3 Performance Analysis Data Sources

The performance acceptability analysis in this paper is based on the following literature data:

- Gartner 2024 survey [18]: 67% of large enterprises have deployed multiple AI agents, averaging 5–15 per enterprise.
- MetaGPT [1] experimental data: multi-agent collaboration tasks average 8–25 minutes execution time.
- ChatDev [2] experimental data: software development tasks average 15–40 minutes execution time.
- LangChain 2024 report [19]: most common enterprise AI Agent use cases are report generation (35%), data analysis (28%), decision support (20%), customer interaction (12%).
- Nielsen Norman Group research [20]: user tolerance threshold for system initialization operations is approximately 10 seconds.
- Google AI UX Guidelines [21]: user tolerance for first-response latency in AI Agent tasks is positively correlated with task complexity.

Based on the above data, this paper categorizes enterprise agent tasks by execution time into four types (report generation, data analysis, decision support, real-time interaction), analyzing each type's tolerance for initialization latency.

References

1. Hong, S., Zhuge, M., Chen, J., et al.: MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In: ICLR 2024 (2024)
2. Qian, C., Liu, W., Liu, H., et al.: ChatDev: Communicative Agents for Software Development. In: ACL 2024 (2024)
3. Guo, Y., Kang, Z., Liu, J., et al.: Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control. arXiv:2505.18279 (2025)
4. Kasselman, P., Lombardo, J.-F., Rosomakho, Y., Campbell, B.: AI Agent Authentication and Authorization (AIMS). IETF Internet-Draft, draft-klrc-aiagent-auth-00 (2026)
5. Cruz, A.: Agent Authorization Profile (AAP) for OAuth 2.0. IETF Internet-Draft, draft-aap-oauth-profile-00 (2026)
6. Sandhu, R.S., et al.: Role-Based Access Control Models. IEEE Computer **29**(2), 38–47 (1996)
7. Jin, X., Krishnan, R., Sandhu, R.: A Unified Attribute-Based Access Control Model. In: IEEE ICWS 2012 (2012)
8. Chen, Z., Lin, B., Wang, J., et al.: AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In: NeurIPS 2024 (2024)
9. Wang, Y., Tong, Y., Li, Y., et al.: BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents. arXiv:2406.03007 (2024)
10. Li, Y., Chen, G., et al.: MemoryScope: Long-Term Memory in LLM Agents. arXiv (2025)

11. Wei, Q., Yang, T., Wang, X., et al.: A-MemGuard: A Proactive Defense Framework for LLM-Based Agent Memory. arXiv:2510.02373 (2025)
12. Prakash, S.: Agent Identity Protocol (AIP). IETF Internet-Draft (2026)
13. Groth, J.: On the Size of Pairing-based Non-interactive Arguments. In: EUROCRYPT 2016, pp. 305–326 (2016)
14. Wang, B., He, Y., Zeng, Z., et al.: Unveiling Privacy Risks in LLM Agent Memory. In: ACL 2025 (2025)
15. Camenisch, J., Lysyanskaya, A.: An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation. In: EUROCRYPT 2002, pp. 308–323 (2002)
16. Bobolz, J., Eidens, F., Krenn, S., Ramacher, S., Samelin, K.: Issuer-Hiding Attribute-Based Credentials. In: CANS 2021, LNCS 13099, pp. 158–178 (2021)
17. Baldimtsi, F., Chalkias, K., et al.: zkLogin: Privacy-Preserving Blockchain Authentication with Existing Credentials. arXiv:2401.11735 (2024)
18. Gartner: AI Agent Adoption in Enterprise: 2024 Survey. Gartner Research (2024)
19. LangChain: State of AI Agents 2024. LangChain Blog (2024)
20. Nielsen, J.: Response Times: The 3 Important Limits. Nielsen Norman Group, 1993 (updated 2024)
21. Google: AI UX Guidelines: Designing for AI Agent Interactions. Google Design (2023)